GWANGHUN LIM — AI BACKEND ENGINEER

아키텍처로 말하는 **AI** 백엔드 — Architecture & Stack

금융 도메인에서 멀티에이전트 오케스트레이션 · **RAG** · 온톨로지 추론 시스템을 설계하고 구현합니다. 이 페이지는 이력이 아니라 **설계 결정의 기록**입니다 — 무엇을 택했고, 무엇을 버렸고, 왜 그랬는지.

STACK MAP

레이어로 보는 기술 스택

태그 나열이 아니라, 시스템에서 각 기술이 서 있는 자리 기준으로.

에이전트 오케스트레이션

ORCHESTRATION

LangGraph | **Multi-Agent** | **A2A Protocol** | **MCP** | LangChain | Supervisor / Plan-Execute | HITL

토폴로지 설계(Supervisor·Plan-Execute·Swarm 비교), 에이전트 간 통신(A2A JSON-RPC 2.0·SSE), 도구 연동(MCP 서버 직접 구축)

지식 · 검색

KNOWLEDGE & RAG

RAG | **Ontology (OWL 2 DL)** | SWRL | SPARQL | Apache Jena / Fuseki | LightRAG (Graph RAG) | pgvector | FAISS

벡터 검색과 심볼릭 추론(온톨로지·룰엔진)을 각각의 적합 지점에 배치 — "임베딩으로 찾고, 온톨로지로 검증한다"

평가 · 관측

EVALUATION & OBSERVABILITY

RAGAS | **DeepEval** | LangSmith | HermiT Reasoner

"동작한다"와 "검증됐다"를 구분 — 정량 지표 기반 자동 평가 파이프라인과 전 구간 트레이싱

서비스 · 데이터

SERVING & DATA

Python / FastAPI | PostgreSQL | Redis | MSA | asyncio | SSE Streaming | Spring Boot

비동기 병렬·캐싱·스트리밍으로 에이전트 지연을 서비스 수준으로 — Redis(휘발) + PostgreSQL(영구) 하이브리드 스토어

인프라

INFRA

Docker | Kubernetes | AWS | CI/CD | On-premise 금융 폐쇄망

클라우드 배포와 규제 금융 폐쇄망 — 양쪽 제약을 모두 아는 상태로 설계

PROJECT 01

로보 어드바이저 — 멀티에이전트 투자 자문 시스템 AWS 배포

요청 분류·라우팅(Supervisor)과 단계 실행(Plan-Execute)을 결합한 하이브리드 토폴로지. 에이전트는 독립 마이크로서비스로 분리되어 A2A 프로토콜로 통신하고, 도구 접근은 전부 MCP 서버로 표준화했다.



asyncio 병렬 호출로 Market Research · Robo Advisor 동시 실행 — 동기 순차 대비 응답 지연 약 50% 단축

DESIGN DECISIONS

Supervisor 단독 대신 하이브리드 토폴로지

순수 Supervisor는 다단계 작업에서 왕복이 늘고, 순수 Plan-Execute는 동적 라우팅이 약하다. 분류·라우팅은 Supervisor, 복잡 요청은 Plan 단계를 거치는 하이브리드로 결합.

에이전트 = 독립 마이크로서비스

단일 프로세스 그래프 대신 서비스 단위 분리. 장애 격리와 개별 배포를 얻는 대신 통신 규약이 필요해져 — A2A(JSON-RPC 2.0)로 표준화.

도구 접근은 전부 **MCP**로

에이전트가 API를 직접 호출하지 않고 도메인별 MCP 서버를 경유. 도구 교체·권한 통제·감사가 한 지점에 모인다.

Redis + PostgreSQL 하이브리드 스토어

대화 컨텍스트·Task 추적은 휘발성 Redis, 영구 기록은 PostgreSQL. 단일 저장소의 단순함 대신 각 데이터의 수명에 맞는 비용·속도를 택함.

STACK

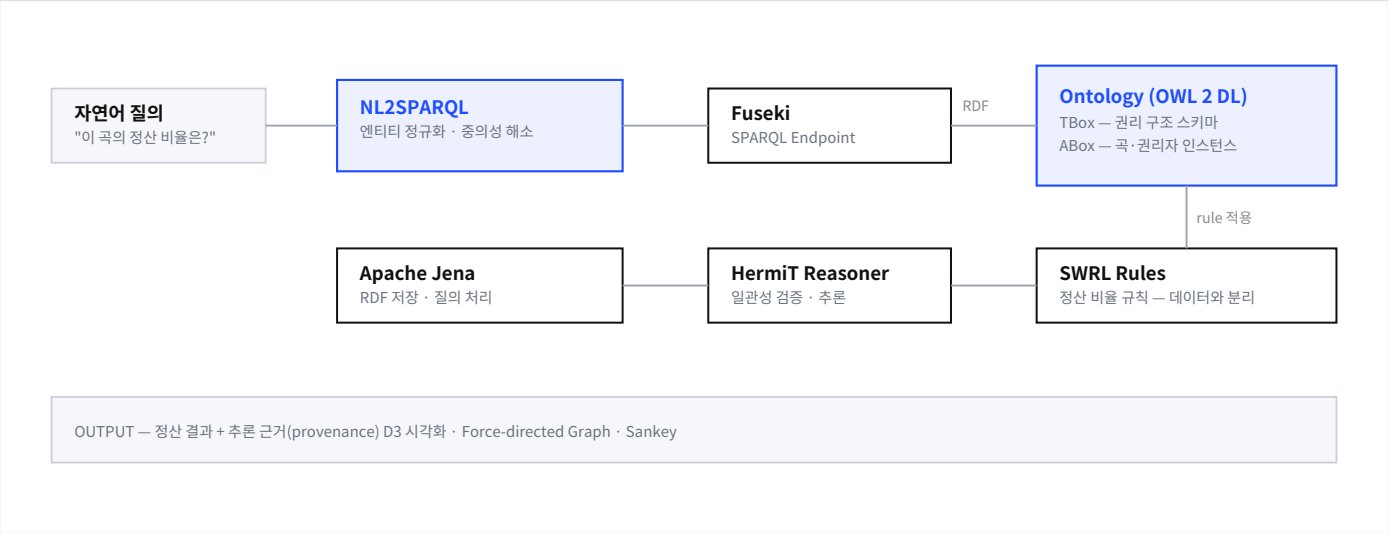
LangGraph · A2A · MCP (FastMCP)
FastAPI · asyncio · SSE
Redis · PostgreSQL · FAISS
LangSmith · Docker · Kubernetes
AWS

PROJECT 02

온톨로지 기반 권리관계 추론 · 정산 룰엔진

진행 중

음악 저작권 도메인의 권리관계를 OWL 2 DL 온톨로지로 모델링하고, 정산 규칙을 SWRL 룰로 분리했다. LLM이 확률적으로 답하는 대신, 자연어 질의를 SPARQL로 변환해 추론 근거가 추적 가능한 심볼릭 레이어에서 답한다.



LLM은 질의 변환까지만 — 답은 온톨로지가 낸다. 정산처럼 "왜 이 금액인가"를 설명해야 하는 도메인의 설계.

DESIGN DECISIONS

RAG 대신 온톨로지 + 룰엔진

정산은 근사가 아니라 정확·설명가능해야 한다. 임베딩 검색의 확률적 답 대신, 스키마(TBox)·사실(ABox)·규칙(SWRL)을 분리한 심볼릭 추론을 택함.

정산 비율은 **ABox** 사실이 아니라 **SWRL** 룰

비율을 인스턴스 데이터로 박으면 규칙 변경 = 데이터 전면 수정. 규칙을 룰 레이어로 분리해 정책 변경이 데이터에 손대지 않게.

NL2SPARQL에 엔티티 중의성 해소 단계

동명 곡·동명 권리자가 흔한 도메인. LLM 변환 전에 정규화·후보 축소를 두어 잘못된 엔티티로 질의가 나가는 걸 차단.

근거 추적을 1급 시민으로

모든 정산 결과에 "어느 규칙·어느 관계를 거쳤나"를 D3 그래프로 노출 — 신뢰가 기능인 도메인의 요구사항.

STACK

OWL 2 DL · SWRL · SPARQL · Apache Jena · Fuseki · HermiT · Protégé · NL2SPARQL · D3.js

PROJECT 03

LightRAG 기반 RAG 성능 비교 프레임워크

실험 완료

"어떤 RAG가 더 좋은가"를 감이 아니라 지표로 답하기 위한 평가 하네스. 그래프 기반 LightRAG와 커스텀 청크 파이프라인을 동일 코퍼스·동일 질의셋에서 RAGAS·DeepEval 지표로 비교했다.



같은 질문, 같은 문서, 다른 파이프라인 — 차이는 지표로만 말한다.

DESIGN DECISIONS

평가를 파이프라인과 분리된 하네스로

각 RAG 구현 안에 평가를 심으면 조건이 미묘하게 어긋난다. 코퍼스·질의셋·실행을 하네스가 소유하고 파이프라인은 교체 가능한 플러그인으로.

RAGAS와 DeepEval 병행

RAGAS는 RAG 특화 표준 지표(faithfulness 등), DeepEval은 도메인 커스텀 메트릭 정의가 유연 — 표준 비교와 도메인 요구를 각각 담당.

그래프 RAG를 무조건 승자로 두지 않음

LightRAG는 관계 질의에 강하지만 색인 비용·지연이 붙는다. 질의 유형별로 승자가 갈린다는 걸 지표로 확인하는 것 자체가 목적.

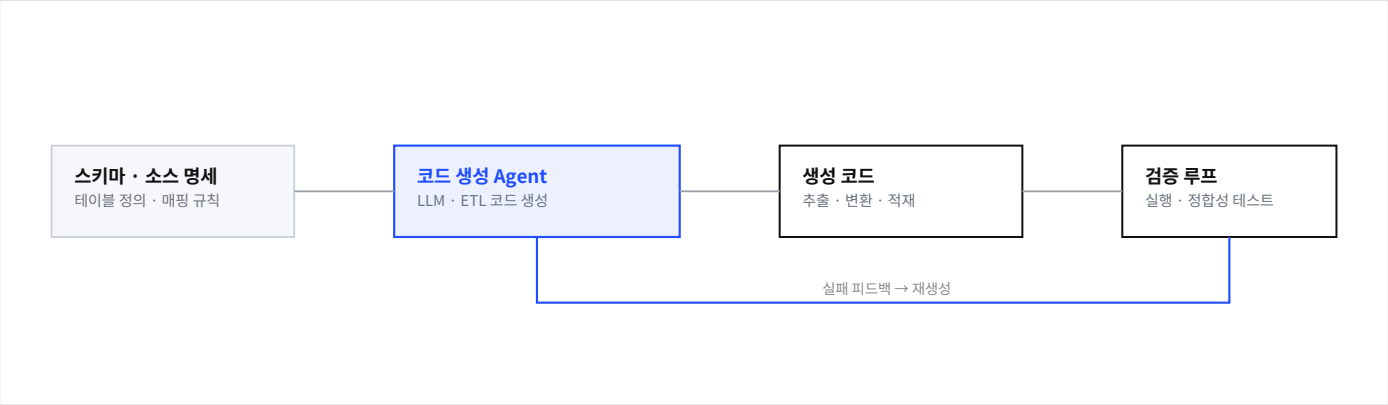
STACK

LightRAG · RAGAS · DeepEval · pgvector · Python · Graph RAG

PROJECT 04

gen-ETL — ETL 코드 생성 에이전트 진행 중단

소스·타깃 스키마 명세로부터 ETL 코드를 생성하고, 실행·검증 결과를 피드백 루프로 되돌리는 에이전트. 설계 검증 단계에서 중단했지만, 중단 판단 자체가 이 프로젝트에서 가장 값진 산출물이었다.



status: halted — 아래 "중단에서 배운 것"이 이 카드의 본문이다.

WHY HALTED — 중단에서 배운 것

검증 비용이 생성 비용을 넘어섰다

코드 생성은 쉽지만, 생성된 ETL의 정합성을 자동 검증하는 하네스가 사실상 본체였다. 검증 인프라 없는 코드 생성은 리뷰 부담을 이동시킬 뿐이라는 판단.

중단은 실패가 아니라 설계 판단

"에이전트가 만든 산출물은 검증 체계가 먼저"라는 원칙은 이후 RAG 평가 프레임워크(03)와 RA의 평가 파이프라인 설계로 직결됐다.

STACK

Python · LangChain · LLM Codegen · ETL